ЧИСЛЕННЫЕ МЕТОДЫ

Основы MATLAB

Юдинцев В. В.

19 сентября 2015 г.

Кафедра теоретической механики
Самарский государственный аэрокосмический университет
им. академика С. П. Королёва
(национальный исследовательский университет)

http://yudintsev.info

СОДЕРЖАНИЕ

Интерполирование табличных данных

Численное дифференцирование

Численное интегрирование

Интегрирование ОДУ

Пример: задача баллистики

Управление процессом интегрирования

ИНТЕРПОЛИРОВАНИЕ ТАБЛИЧНЫХ ДАННЫХ

ПОЛИНОМИАЛЬНАЯ ИНТЕРПОЛЯЦИЯ

- Известны значения некоторой функции $y(x)$ в точках $x_0, x_1, x_2, \ldots, x_n$:
$$y_0, y_1, y_2, \ldots, y_n.$$
- Необходимо построить интерполирующий многочлен $L_n(x)$, совпадающий с значениями табличной функции в точках $x_0, x_1, x_2, \ldots, x_n$ и приближающий функцию $y(x)$ на интервале $[x_0, x_n]$.

Интерполирующий многочлен в форме Лагранжа:

$$L_n = \sum_{i=0}^{n} y_i \frac{(x - x_0)(x - x_1)\ldots(x - x_{i-1})(x - x_{i+1})\ldots(x - x_n)}{(x_i - x_0)(x_i - x_1)\ldots(x_i - x_{i-1})(x_i - x_{i+1})\ldots(x_i - x_n)}$$

ПОЛИНОМИАЛЬНАЯ ИНТЕРПОЛЯЦИЯ: ФУНКЦИЯ POLYFIT

Исходные данные

```
x = [0.1 0.2 0.3 0.4];
y = [0.2 0.6 0.5 0.3];
```

Вычисление коэффициентов интерполирующего многочлена $\tilde{y} = p_n x^n + p_{n-1} x^{n-1} + \ldots + p_0$ для $n = 3$

```
p = polyfit(x,y,3)
ans =
    66.6667   -65.0000    18.8333    -1.1000
```

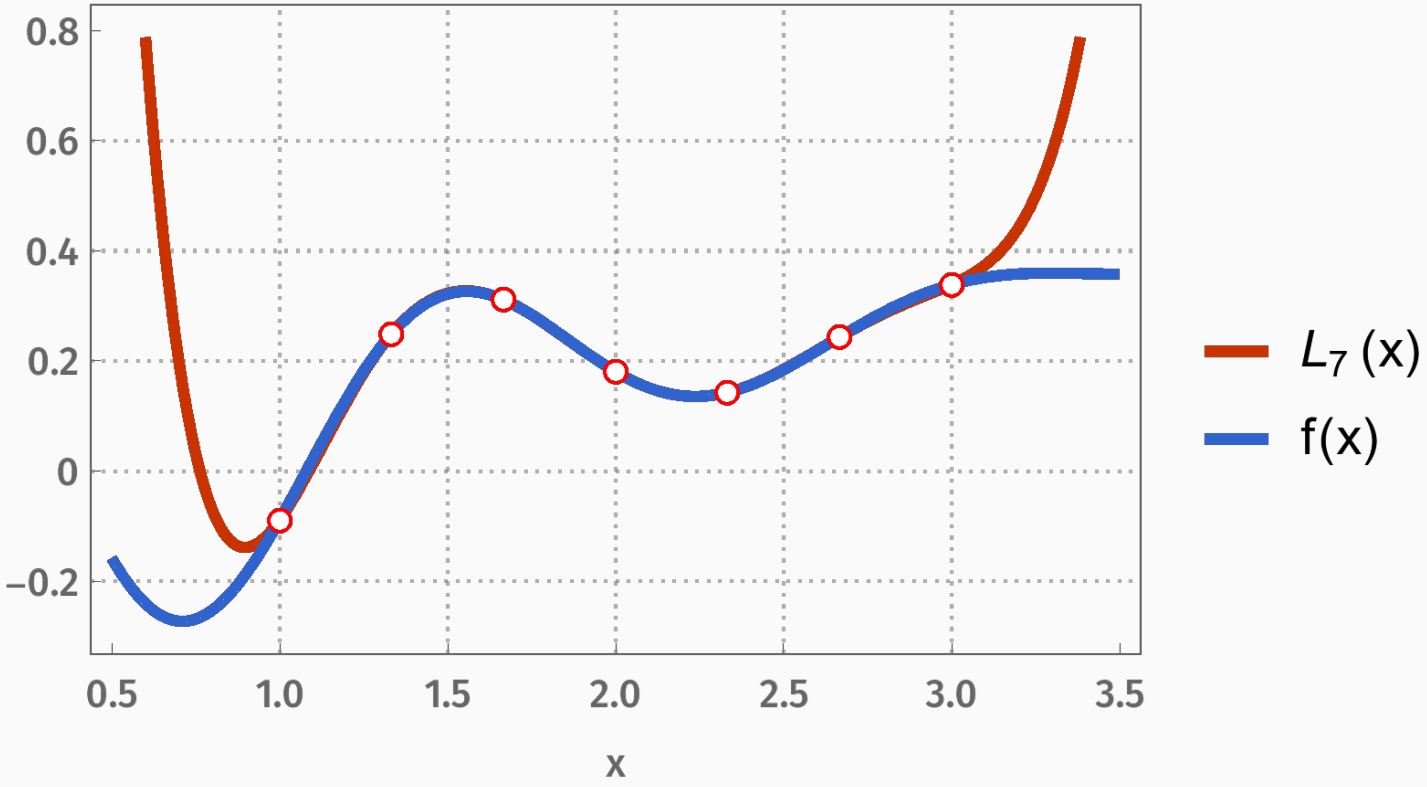
Первый элемент – p_n, последний – p_0.

ВЫЧИСЛЕНИЕ ЗНАЧЕНИЯ ИНТЕРПОЛИРУЮЩЕЙ ФУНКЦИИ

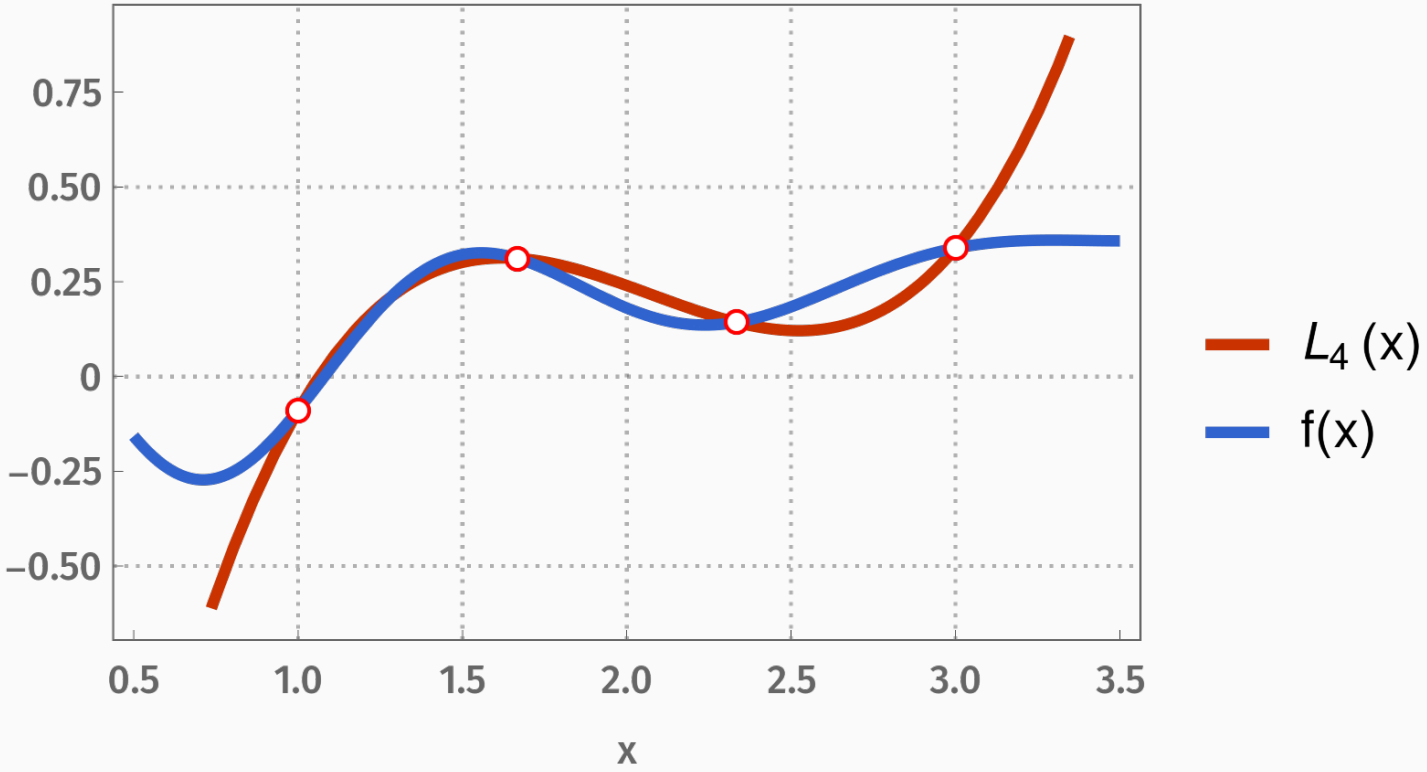
Значения многочлена в точке $x = 0.15$:

```
>> polyval(p,0.15)
ans =
      0.48750
```

ПОЛИНОМИАЛЬНАЯ ИНТЕРПОЛЯЦИЯ

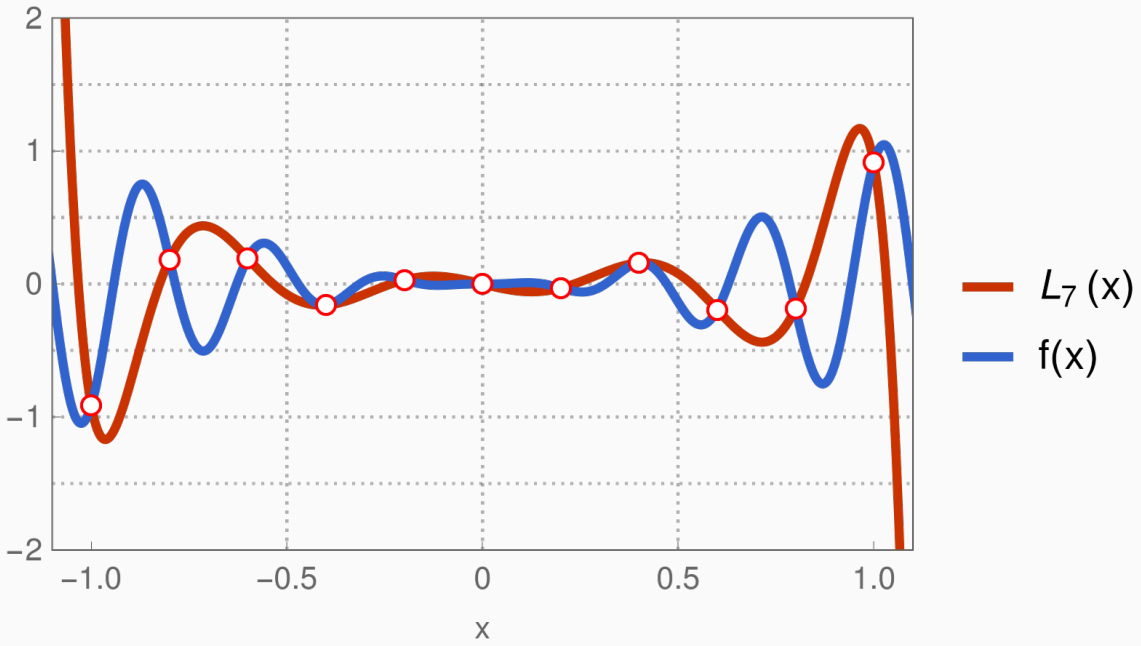


ПОЛИНОМИАЛЬНАЯ ИНТЕРПОЛЯЦИЯ



ПОЛИНОМИАЛЬНАЯ ИНТЕРПОЛЯЦИЯ

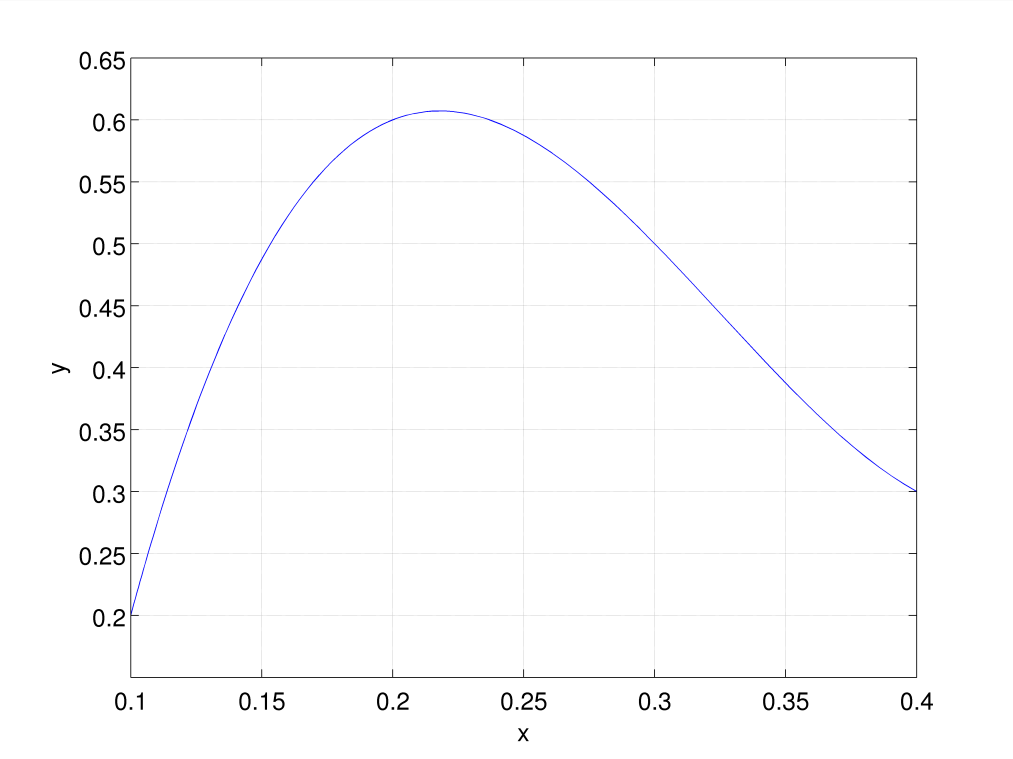
- Полиномиальная интерполяция используется локально.



- Для глобальной интерполяции используется сплайн-интерполяция – глобальная „кусочно-полиномиальная“ интерполяция.

ПОСТРОЕНИЕ ГРАФИКА

```
>> fplot(@(x) polyval(p,x),[0.1 0.4]);
>> grid on;
>> xlabel('x');ylabel('y');
>> print('-dpng', '-r600', 'function.png');
```



Для числа или вектора/матрицы xp: линейная интерполяция

```
yp = interp1(x,y,xp)
```

поиск ближайшего табличного значения

```
yp = interp1(x,y,xp,'nearest')
```

сплайн-интерполяция (полином 3 степени)

```
yp = interp1(x,y,xp,'spline')
```

ЧИСЛЕННОЕ ДИФФЕРЕНЦИРОВАНИЕ

Задана таблица значений $x_0, x_1, x_2, \ldots, x_n$:

```
>> x = [0.1  0.21  0.32  0.41];
```

Конечные разности 1-го порядка

$$\Delta x_k = x_{k+1} - x_k, \; k = 0, \ldots, n-1$$

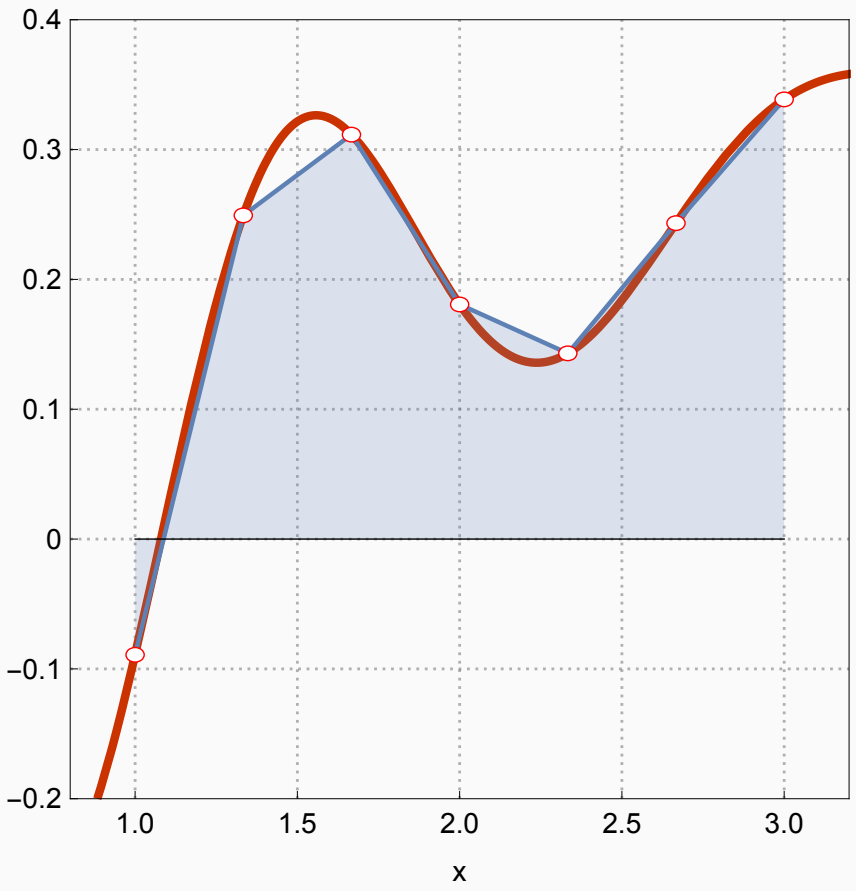
```
>> dx = diff(x)
ans =
      0.110000    0.110000    0.090000
```

Конечная разности 2-го порядка

```
>> dx = diff(x,2)
ans =
     2.7756e-17   -2.0000e-02
```

ЧИСЛЕННОЕ ИНТЕГРИРОВАНИЕ

МЕТОД ТРАПЕЦИЙ



Табличная функция:

```
>> x = [0.1 0.21 0.32 0.41];
>> y = sin(x)+x;
```

Вычисление интеграла:

```
>> trapz(x,y)
ans =
    0.1569
```

$$\int_a^b y(x)dx \approx \frac{1}{2}\sum_{i=1}^{n}(x_{n+1}-x_n)(y_n+y_{n+1})$$

Интегрирование методом Симпсона

```
integral = quad(@function,a,b,eps);
```

`a,b` – пределы интегрирования

`eps` – точность (по умолчанию 10^{-6})

`@function` – ссылка на функцию или строка-выражение, например, `'sin(x)+x'`.

ИНТЕГРИРОВАНИЕ ОДУ

$$\begin{cases} \dfrac{dq_1}{dt} = f_1(t, q_1, q_2, \ldots, q_n), \\ \dfrac{dq_2}{dt} = f_2(t, q_1, q_2, \ldots, q_n), \\ \ldots \\ \dfrac{dq_n}{dt} = f_n(t, q_1, q_2, \ldots, q_n). \end{cases} \quad \Rightarrow \quad \frac{d\boldsymbol{q}}{dt} = \boldsymbol{f}(t, \boldsymbol{q})$$

где

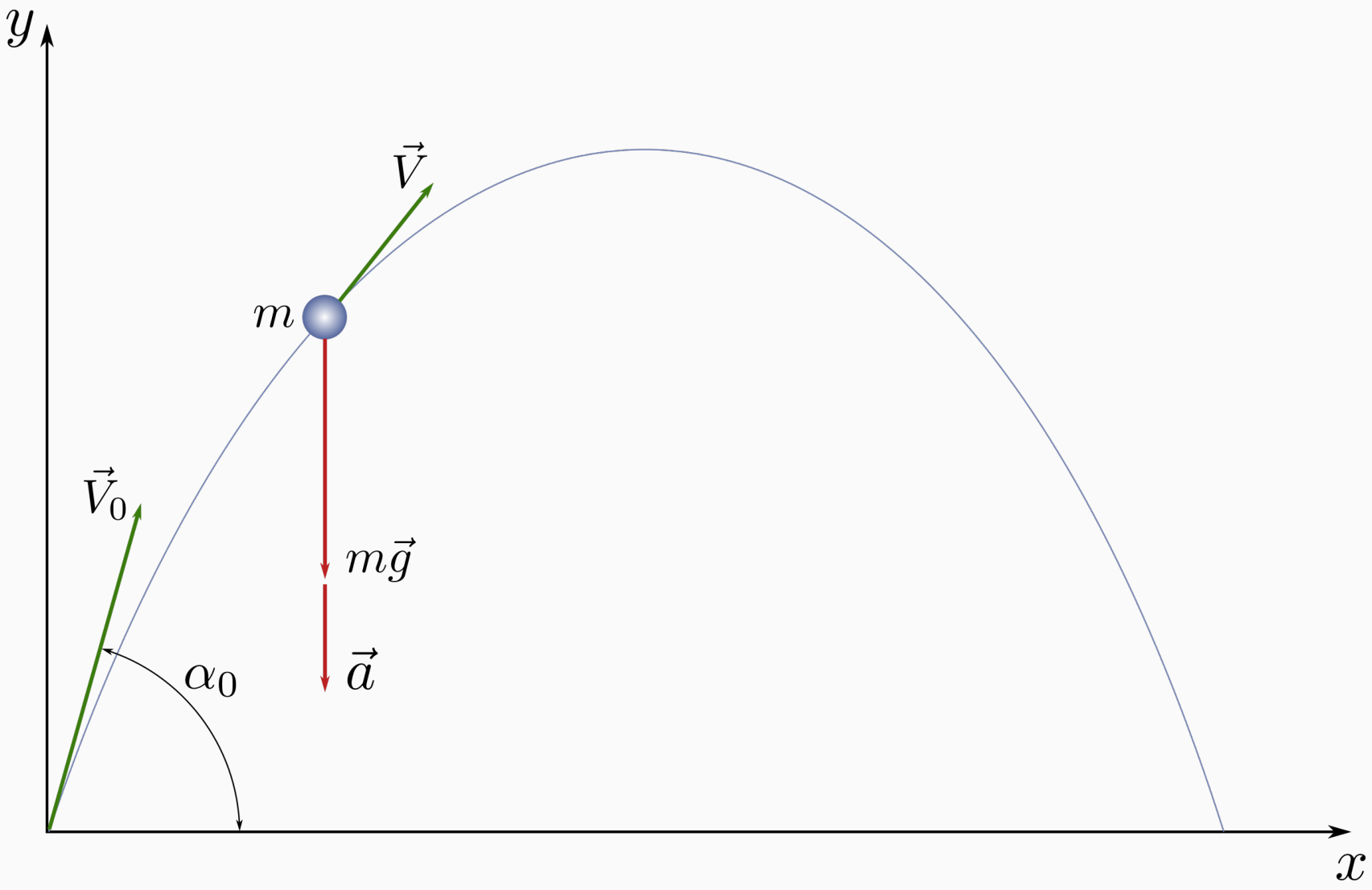
- $\boldsymbol{q} = (q_1, q_2, \ldots, q_n)$ – вектор состояния;
- $\boldsymbol{f}(t, \boldsymbol{q})$ – вектор-функция правых частей.

СТРУКТУРА

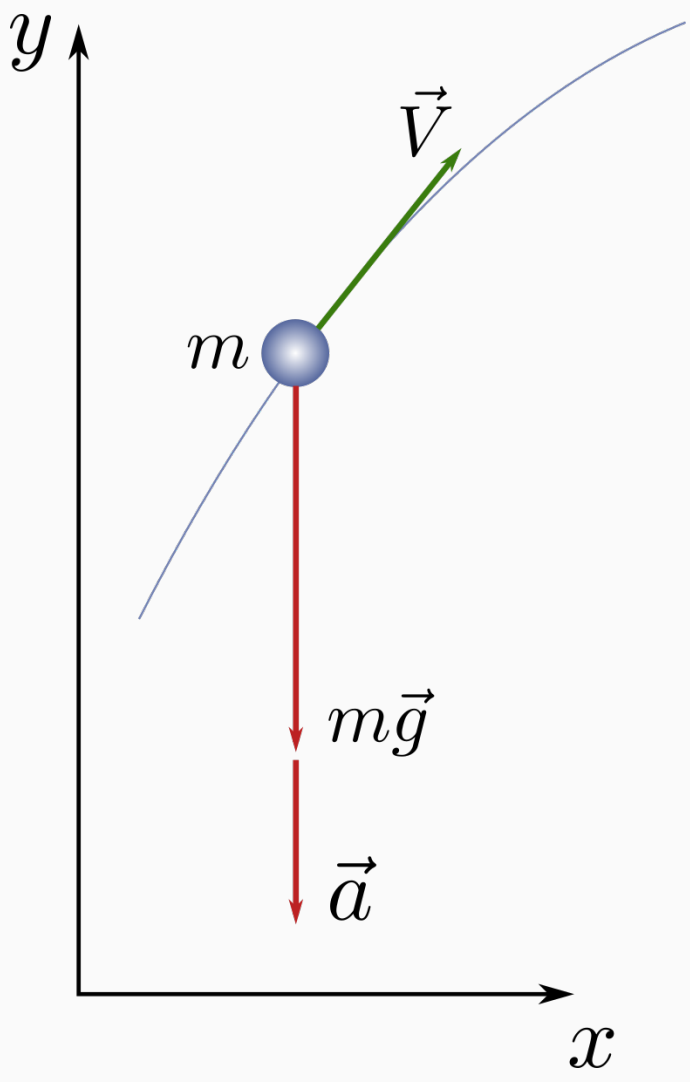
- Приведение к системе дифференциальных уравнений первого порядка.
- Файл-функция правой части $\boldsymbol{f}(t, \boldsymbol{q})$.
- Файл-скрипт запуска процесса интегрирвования ДУ.

ПРИМЕР: ЗАДАЧА БАЛЛИСТИКИ

ЗАДАЧА БАЛЛИСТИКИ



УРАВНЕНИЯ ДВИЖЕНИЯ



Векторное уравнение

$$m\vec{a} = \vec{F}_g = m\vec{g}.$$

Проекции векторного уравнения на оси координат:

$$ma_x = m\frac{d^2x}{dt^2} = 0,$$
$$ma_y = m\frac{d^2y}{dt^2} = -mg.$$

Приведение к системе дифференциальных уравнений первого порядка.

$$\begin{cases} m\dfrac{d^2x}{dt^2} = 0, \\ m\dfrac{d^2y}{dt^2} = -mg. \end{cases} \quad \Rightarrow \quad \begin{cases} \dfrac{dx}{dt} = V_x, \\ \dfrac{dV_x}{dt} = 0, \\ \dfrac{dy}{dt} = V_y, \\ \dfrac{dV_y}{dt} = -g. \end{cases} \quad \Rightarrow \quad \frac{d\boldsymbol{q}}{dt} = \boldsymbol{f}(\boldsymbol{q}, t)$$

где

$$\boldsymbol{q} = (x, V_x, y, V_y), \quad \boldsymbol{f}(\boldsymbol{q}, t) = (V_x, 0, V_y, -g)$$

ODE ФАЙЛ – ФАЙЛ-ФУНКЦИЯ ПРАВЫХ ЧАСТЕЙ

Файл в котором вычисляется правая часть дифференциального уравнения или системы уравнений:

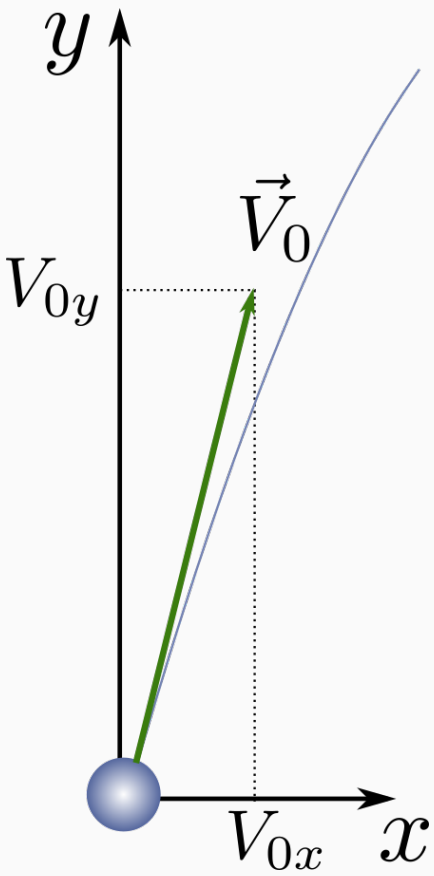
$$\mathbf{q} \Rightarrow \boxed{\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, t)} \Rightarrow \dot{\mathbf{q}}$$

где $\mathbf{q}$ – вектор зависимых переменных, t – время.

Файл-функция `dqdt.m`

```
function dq = dqdt(t,q)
        g = 9.81;
        m = 1.00;
        x=q(1); Vx=q(2); y=q(3); Vy=q(4);
        dq = [Vx, 0, Vy, -g];
```

ИНТЕГРИРОВАНИЕ




```
% Начальные условия
V0 = 10;
a  = pi/4;
q0 = [0, V0*cos(a), 0, V0*sin(a)];
% Интервал интегрирования
tspan = [0 5];
[t, q] = ode45(@dqdt, tspan, q0);
```

Интегратор	Описание
ode45	Метод Рунге-Кутты 4 и 5 порядков с автоматическим выбором шага
ode23	Метод Рунге-Кутты 2 и 3 порядков с автоматическим выбором шага
ode113	Многошаговый метод Адамса: метод с предсказанием и коррекцией
ode15s	Многошаговый метод для интегрирования жестких систем
ode23s	Одношаговый метод Розенброка второго порядка для жестких систем

РЕЗУЛЬТАТЫ ИНТЕГРИРОВАНИЯ

```
[t, q] = ode45(@dqdt, tspan, q0);
```

t – столбец независимой переменной (время)

$$t = \begin{bmatrix} t_0 \\ t_1 \\ \dots \\ t_k \end{bmatrix}$$

q – матрица зависимых переменных

$$q = \begin{bmatrix} x_0 & V_{x0} & y_0 & V_{y0} \\ x_1 & V_{x1} & y_1 & V_{y1} \\ \dots & & & \\ x_k & V_{xk} & y_k & V_{yk} \end{bmatrix}$$

УПРАВЛЕНИЕ ПРОЦЕССОМ ИНТЕГРИРОВАНИЯ

Расширенная версия запуска интегрирования д.у.:

```
[t, q] = ode45( @dqdt, tspan, q0, options);
```

До запуска интегрирования заполняется специальная структура `options` при помощи функции `odeset`:

```
options = odeset('Парам. 1', Значение, 'Парам. 2', ...);
[t, q] = ode45( @dqdt, tspan, q0, options);
```

ТОЧНОСТЬ РЕШЕНИЯ

- `'RelTol'`: относительная погрешность. **Число** - значение погрешности для всех переменных, или **вектор**, задающий погрешность для каждой переменной (по умолчанию 10^{-3})
- `'AbsTol'`: абсолютная погрешность. Число или вектор (по умолчанию 10^{-6})

```
options = odeset('RelTol',1e-4,'AbsTol',1e-8);
[t, q] = ode45( @dqdt, tspan , q0, options );
```

- `'InitialStep'`: начальный шаг интегрирования;
- `'Mass'`: матрица коэффициентов $\mathbf{M}$ в дифференциальных уравнениях, записанных в виде

$$\mathbf{M}(\mathbf{q}, t)\dot{\mathbf{q}} = \mathbf{f}(t, \mathbf{q}).$$

  Имя переменной, если матрица масс постоянная, или ссылка на m-файл. Для всех интеграторов кроме `ode15s, ode23t` матрица масс должна быть невырожденная;
- `'MassSingular'`: может ли матрица масс быть вырожденной. Возможные значения: `'yes'`, `'no'`, `'maybe'` (по умолчанию `'maybe'`).

СОБЫТИЯ: ПАРАМЕТР 'EVENTS'

Для остановки процесса интегрирования при выполении некоторого условия используется функция событий, которая передаётся интегратору:

```
options = odeset('Events',@events);
```

```
V0 = 1;
a  = pi/4;
q01 = [0,V0*cos(a),0,V0*sin(a)];
[t1,q1] = ode45(@dqdt,[0 20],q01);
```

ПРИМЕР ФАЙЛ-ФУНКЦИИ СОБЫТИЙ EVENTS.M

$$q = (x, V_x, y, V_y)^T$$

```
function [ value, isterminal, direction ] = events(t, q)
 % Следим за изменением переменной y
 value = q(3);
 % Остановить процесс,
 isterminal= 1;
 % если q(3) станет равна нулю и будет убывать
 direction =-1;
```

Файл-функция для остановки процесса интегрирования (isterminal=1) при пересечении нуля функцией $y(t)$ (value = q(3)) при её убывании (direction = -1).

```
options = odeset('Events',@events);
v0 = 1;
q01 = [0,v0*cos(pi/4),0,v0*sin(pi/4)];
[t1,q1] = ode45(@dqdt,[0 20],q01);
```

```
n1 = length(t1);
```

Перед интегрированием второго этапа меняем знак скорости V_y (шарик отскочил с в два раза меньшей скоростью):

```
q02    =   q1(n1,:);
q02(4)= -q02(4)*0.5;
```

Интегрируем от t_{1k} до $t_{1k} + 20$

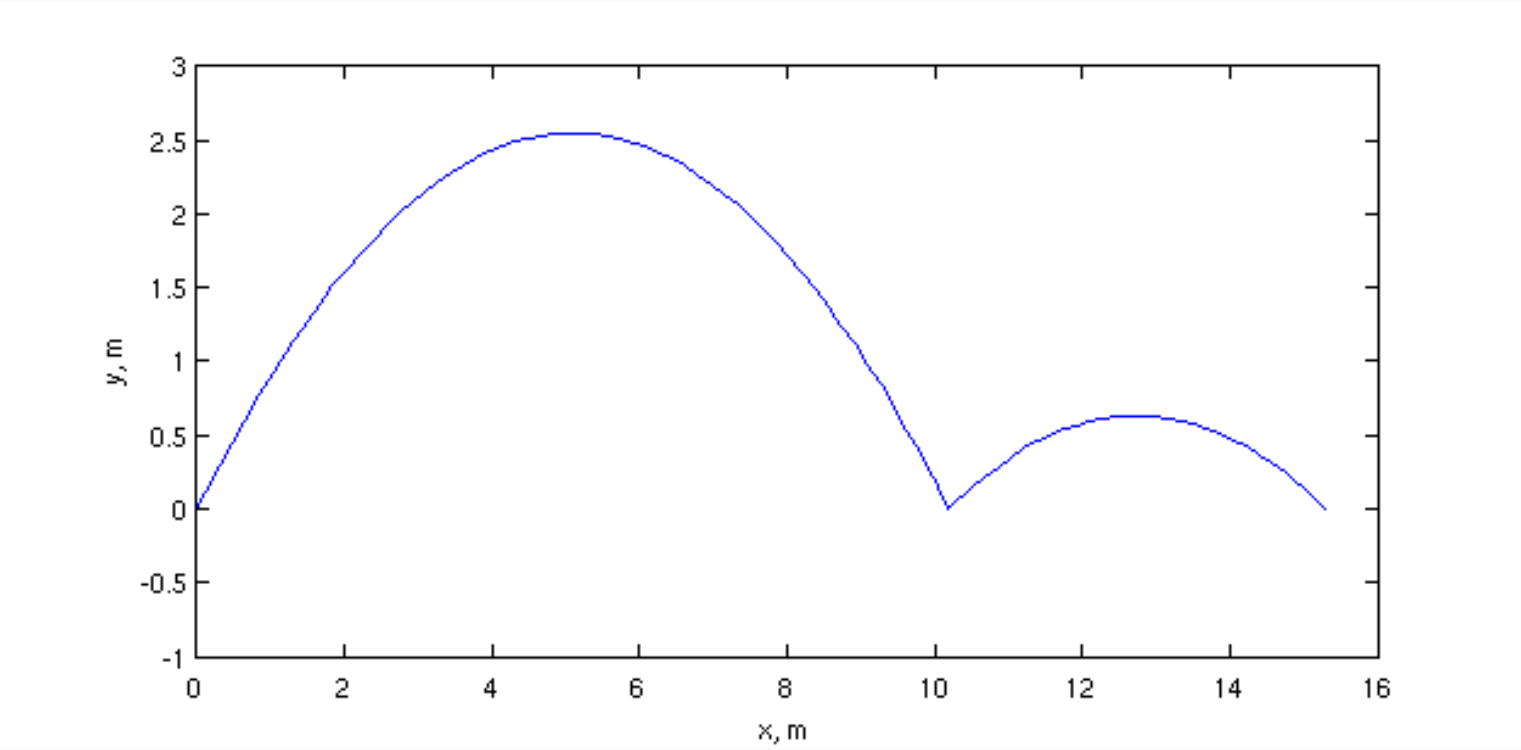
```
t0 = t1(n1); tk=t0+20;
[t2,q2] = ode45(@dqdt,[t0 tk],q02);
```

“Склеиваем” результаты двух этапов движения:

```
t=[t1;t2]; q=[q1;q2];
```

Построение траектории движения шарика:

```
plot(q(:,1),q(:,3));
```



Vadim Yudintsev
Computational Mechanics with MATLAB
Lecture 3: Numerical methods

LaTeX Beamer theme METROPOLIS
by Matthias Vogelgesang

https://github.com/matze/mtheme